

- RAG Learning

4.7 Computer Organisation & Architecture

Computer Science — AQA A-Level (7517) — Paper 2: Written Theory

Topic 4.7 is where the abstract machine of 4.5 and the silicon of 4.6 meet the flesh-and-blood computer sitting on your desk. This note walks every subsection of the spec in order: the internal hardware, von Neumann's stored-program insight, the processor's internal structure, the register set, the instruction set, the fetch-decode-execute cycle, the factors that govern CPU performance, secondary storage, virtual storage, the RISC/CISC divide, the parallel world of GPUs, input/output devices and a practical foothold in assembly language programming. The material is dense but each section connects to the next — follow the signal from instruction in RAM, through the bus, into the CPU, round the cycle, and back out again.

4.7.1 Internal Hardware Components of a Computer

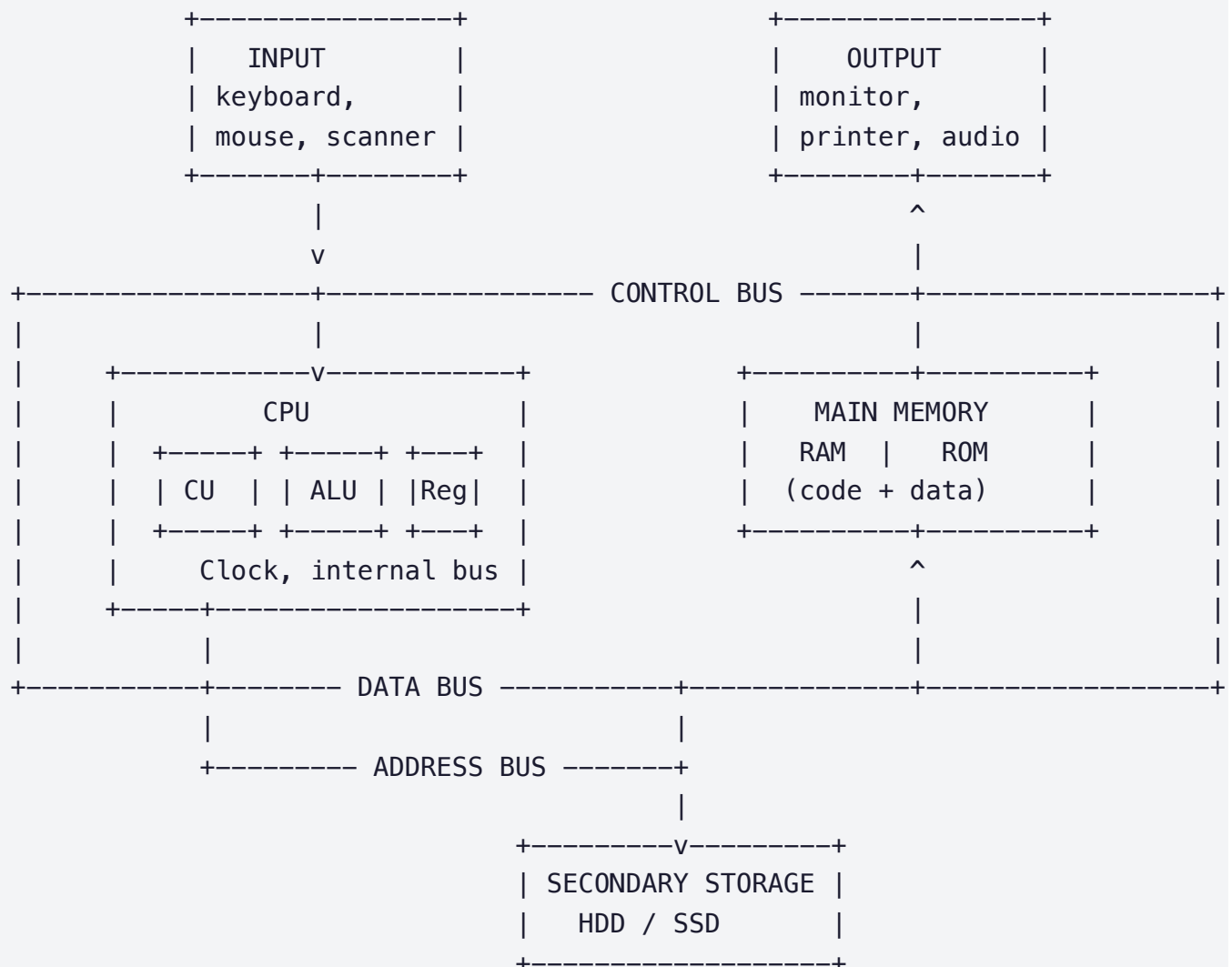
A modern computer is assembled from a small number of functional blocks linked by shared electrical highways called **buses**. The same block diagram describes a smartwatch and a supercomputer — the differences are in scale, not structure.

Central Processing Unit (CPU) — the component that fetches instructions from memory, decodes them and executes them. Contains the ALU, control unit, registers and internal buses. The "brain" of the machine; everything else either feeds it or stores what it produces.

- **Processor (CPU)** — executes instructions. See 4.7.3 for internal structure.
- **Main memory** — volatile **RAM** (random-access memory) holds the programs and data currently in use; non-volatile **ROM** (read-only memory) holds firmware such as the BIOS/UEFI boot code.
- **Input/output devices** — keyboards, mice, monitors, printers, network cards. Anything that moves information across the boundary between the computer and the outside world.

- **Secondary storage** — HDDs, SSDs, optical discs, flash drives. Non-volatile, high-capacity, slower than RAM. See 4.7.8 – 4.7.9.
- **Buses** — parallel wires that carry signals between components. Three logical buses ride on one physical motherboard trace bundle:
 - **Address bus** — unidirectional, CPU → memory/IO. Carries the address of the location to access.
 - **Data bus** — bidirectional. Carries the data being read or written.
 - **Control bus** — bidirectional. Carries timing and coordination signals (read, write, clock, interrupt request, bus grant).

Block diagram of a typical computer



Every read from RAM follows the same choreography: the CPU places an address on the address bus, asserts "read" on the control bus, and the addressed memory chip drops the contents onto the data bus for the CPU to collect. A write reverses the data-bus direction and asserts "write" instead.

4.7.2 The Stored Program Concept

Stored program concept — the foundational idea, articulated by John von Neumann in the 1945 *First Draft of a Report on the EDVAC*, that a computer's program instructions are held in the same addressable memory as its data, and that both are represented as binary patterns that the CPU treats interchangeably. The program is loaded into memory just like input data, and the CPU fetches each instruction in turn, executes it, and moves to the next.

Before 1945, general-purpose computing machines (ENIAC, Colossus) were "programmed" by physically rewiring plugboards or flipping rows of switches. Changing the task could take days. Von Neumann's insight collapses the program and its data into a single, uniform representation in memory, which gives modern computing its decisive properties:

- **Flexibility** — loading a new program is just another memory write. No rewiring required.
- **Self-modifying code** — a program can treat instructions as data, which enables compilers, JIT, loaders, and assemblers. Every piece of software you use sits on this.
- **Uniform hardware** — the same CPU runs every task, so mass production is viable.
- **Compilers and operating systems** become possible — a compiler is just a program that emits another program into memory.

The architecture bearing von Neumann's name places instructions and data on a shared bus to one memory; its rival, the **Harvard architecture**, uses separate instruction and data memories (and thus separate buses), allowing the CPU to fetch an instruction and a data word in the same cycle. Modern CPUs are von Neumann at the main-memory level but Harvard at the L1 cache (split I-cache and D-cache) — a hybrid that gives throughput without giving up flexibility.

Exam Tip: Questions about the stored program concept want the phrase "*programs and data share the same memory and are represented in the same binary form*". Also state that

the CPU fetches instructions *sequentially* from this memory unless a branch modifies the PC. Contrast with ENIAC-style plugboards for a comparison mark.

4.7.3 Structure and Role of the Processor

Inside the CPU package are four co-operating sub-systems, wired together by a set of short internal buses and paced by the processor clock.

Arithmetic Logic Unit (ALU) — the combinational circuit that performs arithmetic operations (add, subtract, sometimes multiply/divide), logical operations (AND, OR, NOT, XOR), comparisons, and bit shifts. It has two data inputs, one data output, a set of control lines selecting the operation, and outputs a few status flags (zero, negative, carry, overflow) which are latched in the status register.

Control Unit (CU) — generates the sequence of control signals that drive every other block in the CPU. On each clock tick it decodes the current instruction and asserts the right buses, register enables and ALU opcodes to carry it out. May be *hardwired* (RISC) or *microprogrammed* (CISC).

The register set

A small bank of very fast (single-cycle) storage locations inside the CPU. Some are **special-purpose** (PC, CIR, MAR, MBR, SR, ACC) — their role is fixed by the hardware. Others are **general-purpose** (R0–R15 on ARM, RAX–R15 on x86-64), used by the programmer as scratch locations. More registers means fewer trips to memory, so deeper register files improve performance. See 4.7.4 for a detailed summary.

The clock

A crystal oscillator emits a square-wave pulse at a fixed frequency (e.g. 3.2 GHz = 3.2×10^9 pulses per second). Every internal register update is synchronised to the rising or falling edge of this pulse, so all parts of the CPU advance in lock-step. Faster clock → more stages per second →

more work done, up to the physical limit where signals can no longer propagate across the chip within one period (≈ 5 GHz in silicon).

Internal buses

- **Internal address bus** — connects the MAR to on-chip cache and to the pad driving the external address bus.
- **Internal data bus** — ferries data between the register file, ALU and MBR.
- **Internal control bus** — CU's signal lines (register-enable, ALU-op, read/write).

Pipelining

Pipelining — a hardware optimisation that overlaps the execution of several instructions in time. The fetch-decode-execute cycle is split into stages, and while one instruction is being executed, the next is being decoded and the one after that is being fetched. After the pipeline fills, one instruction *completes* every clock cycle even though each one still takes multiple cycles end-to-end.

A simple **3-stage pipeline** has stages Fetch (F), Decode (D), Execute (E). A more aggressive **5-stage pipeline** (the classic RISC pipeline) is IF (instruction fetch), ID (instruction decode / register read), EX (execute / ALU), MEM (memory access), WB (write back).

3-stage pipeline timing (one stage per cycle):

Cycle:	1	2	3	4	5	6	7
Instr 1:	[F]	[D]	[E]				
Instr 2:		[F]	[D]	[E]			
Instr 3:			[F]	[D]	[E]		
Instr 4:				[F]	[D]	[E]	
Instr 5:					[F]	[D]	[E]

Without pipelining, five instructions take $5 \times 3 = 15$ cycles.

With pipelining, they complete in $3 + (5 - 1) = 7$ cycles.

5-stage pipeline (classic RISC):

Cycle:	1	2	3	4	5	6	7	8
--------	---	---	---	---	---	---	---	---

```

Instr 1:  [IF] [ID] [EX] [MEM] [WB]
Instr 2:           [IF] [ID] [EX] [MEM] [WB]
Instr 3:                   [IF] [ID] [EX] [MEM] [WB]
Instr 4:                           [IF] [ID] [EX] [MEM] [WB]

```

Once a pipeline is full, **throughput** is one instruction per cycle regardless of the number of stages. Deeper pipelines support higher clock speeds (each stage does less, so it finishes faster), but they suffer more on mis-speculation.

Pipeline hazards

Hazard	Cause	Mitigation
Structural	Two stages need the same hardware in one cycle (e.g. both IF and MEM want memory).	Duplicate the unit; split I-cache and D-cache; stall one stage.
Data — RAW (read-after-write)	An instruction reads a register the previous one hasn't yet written.	Forwarding/bypassing; bubble/stall; compiler reorders instructions.
Data — WAR (write-after-read)	An instruction writes a register the previous one is still reading. Only in out-of-order machines.	Register renaming.
Data — WAW (write-after-write)	Two instructions write the same register in the wrong order. Only out-of-order.	Register renaming.
Control (branch)	A branch is decoded too late — the pipeline has already fetched the wrong path.	Branch prediction; delayed branches; flushing on misprediction.

Branch prediction

When the fetch stage meets a conditional branch, the actual direction isn't known until the execute stage resolves the comparison. Rather than stall the pipeline, the CPU **predicts** the direction and

speculatively fetches along the guessed path. If the guess is right, no penalty; if wrong, the pipeline is *flushed* and refilled from the correct target — paying the full pipeline depth in lost cycles. Modern predictors combine *static* hints (backward branches taken, forward not taken) with *dynamic* two-bit saturating counters indexed by branch address, reaching 95%+ accuracy on typical code.

Exam Tip: When asked why pipelining doesn't simply multiply speed by the number of stages, mention three things: (i) the pipeline takes $n-1$ cycles to fill; (ii) hazards force stalls or flushes; (iii) stages must be balanced — the slowest stage sets the clock. Examiners love all three.

4.7.4 Processor Registers

Registers are the tiny, fast storage locations that hold the CPU's working state. Every fetch-decode-execute step reads or writes at least one of them.

Program Counter (PC) — holds the memory address of the *next* instruction to be fetched. Incremented during the fetch phase ($PC \leftarrow PC + 1$ for a one-word instruction) and overwritten wholesale by a branch.

Current Instruction Register (CIR) — holds the instruction currently being decoded and executed. Loaded from the MBR at the end of the fetch phase; read by the CU throughout decode and execute.

Memory Address Register (MAR) — holds the address of the memory location the CPU is about to read from or write to. Its output drives the external address bus.

Memory Buffer Register (MBR) / Memory Data Register (MDR) — holds the data word flowing between the CPU and memory. On a read, the addressed word lands in the MBR; on a write, the CPU loads the MBR and then asserts the write control line.

Status Register (SR) — a collection of 1-bit *flags* automatically set by the ALU after each operation. Read by conditional branches.

- **Z (zero)** — set iff the ALU result was zero.
- **N (negative)** — set iff the result's sign bit (MSB) was 1 (two's-complement negative).
- **C (carry)** — set iff an addition produced a carry out of the MSB (or a subtraction did not borrow). Used for unsigned overflow and multi-word arithmetic.
- **V (overflow)** — set iff a signed operation exceeded the representable range (e.g. positive + positive → negative).

Accumulator (ACC) — on single-accumulator architectures (classic 8-bit CPUs, LMC), the default destination of arithmetic results. Modern RISC CPUs dispense with a dedicated accumulator in favour of many equivalent general-purpose registers.

Register summary

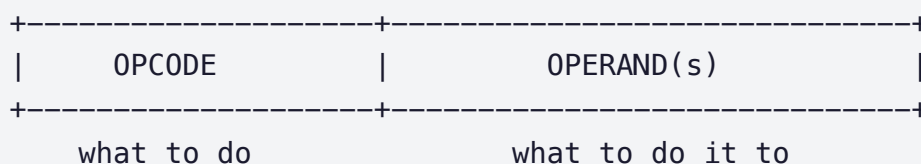
Register	Width	Holds	Updated during
PC	= address bus width	Address of next instruction	Fetch (increment) or execute (branch)
MAR	= address bus width	Address being accessed	Fetch (with PC); execute (with operand)
MBR / MDR	= data bus width	Data to/from memory	Every memory read or write
CIR	= instruction length	Instruction being executed	End of fetch
SR	typically 8 bits	Z, N, C, V flags	After each ALU operation
ACC	= data bus width	Default operand / result	Execute (arithmetic/logical)

R0...Rn (GPRs)	= data bus width	Programmer-controlled scratch	Any LDR/ADD/MOV etc.
-------------------	------------------	----------------------------------	----------------------

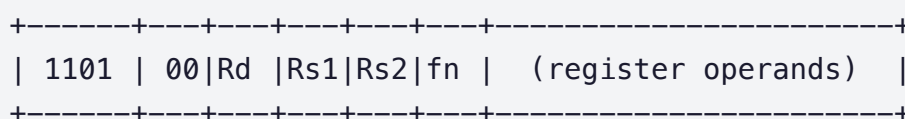
Exam Tip: "What does the MAR contain?" — answer "*the address of the memory location that is about to be read from or written to*", not "some address". Precision wins marks. The MBR "holds the data word being transferred between the CPU and memory" — again, be specific about direction and purpose.

4.7.5 The Processor's Instruction Set

An **instruction set** is the fixed vocabulary of operations the CPU understands. Each instruction is a bit pattern with two fields:



Example ARM-ish encoding (32 bits):



Addressing modes

The **addressing mode** says how the operand field should be interpreted. AQA 7517 examines two modes — immediate and direct; indirect and indexed are shown for wider context:

Mode	Meaning	Example (pseudocode)	Effect
Immediate	Operand is the literal value.	LDR R1, #42	R1 ← 42

Direct (absolute)	Operand is the address of the value.	<code>LDR R1, 200</code>	$R1 \leftarrow \text{contents of memory}[200]$
Indirect	Operand is the address of an address. Follow one level of indirection.	<code>LDR R1, (200)</code>	$R1 \leftarrow \text{memory}[\text{memory}[200]]$
Indexed	Operand is a base address; an index register is added to it.	<code>LDR R1, 200(R2)</code>	$R1 \leftarrow \text{memory}[200 + R2]$

Worked example — suppose $\text{memory}[200] = 300$ and $\text{memory}[300] = 99$, and $R2 = 4$:

```

LDR R1, #42      → R1 = 42          (immediate)
LDR R1, 200      → R1 = 300         (direct: fetch from 200)
LDR R1, (200)    → R1 = 99          (indirect: 200→300→99)
LDR R1, 200(R2)  → R1 = memory[204] (indexed: base 200 + R2=4)

```

Indexed addressing is how arrays are walked: the base address of the array stays put; the index register is simply incremented inside a loop.

4.7.6 The Fetch-Decode-Execute Cycle

Every instruction the CPU runs is carried out by the same three-phase cycle, driven by the control unit and expressed concisely in **register transfer notation** — `destination ← source`, with `[X]` meaning "the contents of X" and `[[X]]` meaning a double dereference.

The three phases in register-transfer notation

FETCH

```

MAR ← [PC]          // copy the next-instruction address to the MAR
PC  ← [PC] + 1      // point PC at the instruction after this one
MBR ← [memory[MAR]] // read from RAM over the data bus
CIR ← [MBR]         // latch the fetched bits into the CIR

```

DECODE

```

decode opcode in CIR          // CU splits opcode and operand fields
                               // and works out the addressing mode
(operand fetched if needed, e.g. MAR ← operand; MBR ← [memory[MAR]])

```

EXECUTE

depending on the opcode:

```

arithmetic: ALU computes on registers & operand; SR flags updated
memory:     read/write via MAR/MBR
branch:     PC ← target address (conditional on SR flags)

```

Trace 1 — **LDR R1, 200** (direct load)

Assume PC = 50, memory[50] = "LDR R1, 200", memory[200] = 99, and R1 is initially 0.

Fetch:

```

MAR ← [PC]          // MAR = 50
PC  ← [PC] + 1      // PC = 51
MBR ← [memory[50]]  // MBR = "LDR R1, 200"
CIR ← [MBR]         // CIR = "LDR R1, 200"

```

Decode:

```
opcode = LDR, Rd = R1, addr = 200, mode = direct
```

Execute:

```

MAR ← 200           // set up the read
MBR ← [memory[200]] // MBR = 99
R1  ← [MBR]         // R1 = 99

```

After the cycle: R1 = 99, PC = 51.

Trace 2 — **ADD R1, R2, R3** (register arithmetic)

Assume PC = 51, memory[51] = "ADD R1, R2, R3", R2 = 7, R3 = 5.

Fetch:

```

MAR ← [PC]          // MAR = 51
PC  ← [PC] + 1      // PC = 52
MBR ← [memory[51]]  // MBR = "ADD R1, R2, R3"
CIR ← [MBR]

```

Decode:

```
opcode = ADD, Rd = R1, Rs1 = R2, Rs2 = R3
```

Execute:

```
ALU.A ← [R2]           // 7
ALU.B ← [R3]           // 5
ALU.op ← ADD
R1 ← ALU.out           // 12
SR: Z=0, N=0, C=0, V=0 // flags updated
```

After the cycle: R1 = 12, PC = 52.

Exam Tip: Register-transfer notation is worth easy marks if you get the syntax right: use the left arrow $\leftarrow$ (not $=$), wrap contents in square brackets ($[PC]$ means "contents of PC") and double them for indirect fetches ($[[MAR]]$). Examiners deduct for $PC = PC + 1$ when they want $PC \leftarrow [PC] + 1$.

4.7.7 Factors Affecting CPU Performance

Real-world CPU performance depends on several interacting parameters. A question asking "how could performance be improved?" expects at least three of the following, each with a mechanism.

Clock speed

Higher frequency $\rightarrow$ more FDE cycles per second $\rightarrow$ more instructions executed per second, *provided* the pipeline is kept full. The hard limit is **power dissipation**: dynamic power scales roughly with $f \times V^2$, so doubling the clock (and the voltage needed to switch reliably at that rate) roughly *octuples* the heat. This is why mainstream clocks have stalled near 5 GHz since 2005 — the industry moved sideways into multi-core instead.

Number of cores

A **core** is a complete CPU: its own ALU, CU, registers, L1 cache. A multi-core chip with N cores can in principle run N threads in parallel, multiplying throughput up to $N\times$. Real speed-up is bounded by **Amdahl's law**: if a fraction p of the work is parallelisable and the rest is serial, maximum speed-up = $1 / ((1-p) + p/N)$. Even with infinite cores, a 10%-serial workload can speed up at most $10\times$.

Cache

Cache — a small, fast block of SRAM sitting between the CPU and main memory. It holds recently- and frequently-accessed addresses; a hit avoids the long trip to DRAM. Organised in levels L1, L2, L3 of increasing size and latency.

Level	Typical size	Latency (cycles)	Scope
L1	32–64 KiB per core (split I/D)	3–5	Private to core
L2	256 KiB – 1 MiB per core	10–20	Private to core
L3	4–64 MiB	30–60	Shared across all cores
Main RAM	8–128 GiB	200–400	Whole system

Worked hit/miss example. Suppose L1 has 2-cycle access with 95% hit rate, and a miss costs 100 cycles to fetch from RAM. Average memory access time (AMAT):

$$\begin{aligned}
 \text{AMAT} &= \text{hit_time} + \text{miss_rate} \times \text{miss_penalty} \\
 &= 2 + 0.05 \times 100 \\
 &= 2 + 5 \\
 &= 7 \text{ cycles}
 \end{aligned}$$

If we improve hit rate to 99%:

$$\text{AMAT} = 2 + 0.01 \times 100 = 3 \text{ cycles} \quad (\text{roughly } 2.3\times \text{ faster})$$

Cache performance exploits two principles: **temporal locality** (a location accessed once is likely to be accessed again soon) and **spatial locality** (its neighbours are likely to be accessed soon). Caches load whole *lines* (typically 64 bytes) to exploit both.

Word length

The **word length** is the number of bits the CPU processes in parallel — typically the width of the data bus, the registers and the ALU. A 64-bit CPU moves and operates on 64 bits at a time, so 64-bit arithmetic takes one cycle rather than many; it also addresses up to 2^{64} bytes directly.

Address bus width

Each extra address line doubles the number of addressable memory locations. With an n -bit address bus you can address 2^n distinct locations.

Worked example: a CPU has a 20-bit address bus.

Number of addressable locations = $2^{20} = 1,048,576 = 1 \text{ MiB}$ (of words).

If each word is 1 byte, maximum RAM = 1 MiB.

If each word is 4 bytes (32 bits), maximum RAM = 4 MiB.

The infamous 640 KiB limit of early IBM PCs came from reserving part of the 1 MiB 20-bit address space for BIOS, video, and expansion ROMs.

Exam Tip: "Calculate the maximum addressable memory" — always $2^{\text{address-bus-width}}$ locations. Multiply by the *word size in bytes* only when the question asks for bytes. Don't forget to state units (bytes, KiB, MiB).

4.7.8 The Need for Secondary Storage

Main memory (RAM) is volatile: cut the power and its contents vanish. It is also expensive per gigabyte. A working computer therefore needs **secondary storage**:

- **Non-volatile persistence** — files, programs, the OS itself must survive a shutdown. Secondary storage retains data indefinitely without power.
- **Capacity** — an SSD in a laptop is typically 50–100× the size of its RAM. Keeping media libraries, installed applications, OS images and user documents in RAM would be neither affordable nor necessary.
- **Portability** — removable devices (USB sticks, optical discs) let data be physically carried or archived.
- **Cost per byte** — ¢/GB for magnetic disks is far below RAM's \$/GB.

4.7.9 Types of Secondary Storage

Magnetic — Hard Disk Drive (HDD)

A stack of rigid glass/aluminium **platters** coated in a ferromagnetic film, spinning at 5,400–15,000 rpm. **Heads** on a moving arm fly a few nanometres above each surface, detecting and reversing the magnetic polarity of microscopic regions — each region encodes one bit. The surface is organised into concentric **tracks**, each subdivided into **sectors** (typically 512 bytes or 4 KiB). Tracks at the same radius across different platters form a **cylinder**, which can be accessed without moving the head.

Access time = seek time (head to track) + rotational latency (sector to arrive) + transfer. Typical 7,200 rpm drive: ~10 ms average seek. Strengths: huge capacity (multi-TB), cheap per GB. Weaknesses: mechanical, fragile under shock, slow, noisy, power-hungry.

Solid-state — SSD

No moving parts. Data is stored as trapped charge on floating-gate transistors arranged in a grid of **NAND flash** cells. A flash controller manages reads, writes and erases in *blocks*. Two important mechanisms:

- **Wear levelling** — each flash cell can only be erased a finite number of times (10^3 – 10^5 cycles). The controller spreads writes uniformly across all cells so no single block fails early.
- **TRIM** — an OS command telling the SSD which blocks the file system has freed. The controller can then erase them ahead of time, keeping write speeds high.

Strengths: very fast (sub-millisecond random reads, GB/s sequential), silent, shock-resistant, low power. Weaknesses: cost/GB higher than HDD, finite write endurance.

Optical — CD / DVD / Blu-ray

Data encoded as microscopic **pits** and **lands** on a spiral track in a reflective metal layer under transparent polycarbonate. A laser shines on the track; pits scatter light and lands reflect it, producing a binary signal. Pit size falls as wavelength shortens: CD uses 780 nm infrared (700 MB), DVD uses 650 nm red (4.7 GB), Blu-ray uses 405 nm blue-violet (25–50 GB). Strengths: cheap per disc, portable, good for archival and distribution. Weaknesses: slow, low capacity by modern standards, scratch-sensitive.

Comparison of secondary-storage technologies

	HDD (magnetic)	SSD (flash)	Optical (DVD/BD)
Speed (random)	~10 ms seek	<0.1 ms	~100 ms (laser seek)
Speed (sequential)	100–200 MB/s	500 MB/s – 7 GB/s	~10–70 MB/s
Capacity (consumer)	1–22 TB	256 GB – 8 TB	4.7–50 GB per disc
Cost per GB	Lowest	Medium	Very low (bulk discs)
Durability	Sensitive to shock	Shock-proof, finite writes	Scratch/heat sensitive
Power draw	6–10 W	2–5 W	~5 W (active only)
Typical use	Bulk storage, servers	OS drive, laptops, phones	Install media, archival

4.7.10 Virtual Storage

Virtual memory (virtual storage) — an OS mechanism that uses part of secondary storage as an overflow extension of RAM. When RAM fills up, inactive pages are *swapped out* to a disk file (page file / swap partition); when the CPU references a swapped-out page, a page fault triggers the OS to swap it back in, evicting another page if necessary.

Virtual storage delivers the illusion of a huge address space and lets more (or larger) programs run than physical RAM would otherwise allow. The trade-off is stark:

- An L1 cache access is ~1 ns.
- A RAM access is ~100 ns.
- An SSD access is ~100 µs (~1000× slower than RAM).
- An HDD access is ~10 ms (~100,000× slower than RAM).

Each page fault therefore costs thousands to millions of CPU cycles. If the working set is only marginally larger than RAM, the system spends most of its time paging rather than computing — this pathology is called **thrashing**. The fix is either to add RAM, reduce the workload, or have the OS's working-set model evict cold pages more aggressively.

Exam Tip: Distinguish *virtual memory* (the OS abstraction presenting each process with its own address space, implemented by paging) from *virtual storage* in this topic's sense (using disk as extra RAM). AQA's wording often treats them interchangeably, but your answer should mention both: pages swapped to the paging file when RAM is exhausted, and the cost being disk latency versus RAM latency.

4.7.11 CISC and RISC

RISC (Reduced Instruction Set Computer) — a design philosophy that keeps the instruction set small, the instructions fixed-length and simple, and the architecture register-rich. Complex tasks are built from sequences of simple instructions. Load/store architecture: memory is touched only by explicit LDR / STR, all other instructions operate on registers. Examples: ARM, RISC-V, MIPS, early SPARC, PowerPC.

CISC (Complex Instruction Set Computer) — a design philosophy that provides a rich, variable-length instruction set, where single instructions can perform multi-step operations (e.g. "add a memory operand to a register and store back to memory"). Historically motivated by small memories and slow compilers: the CPU did more per instruction, so the program was shorter. Examples: x86 / x86-64, VAX, IBM System/360, Motorola 68000.

RISC vs CISC — comparison

	RISC	CISC
Instruction length	Fixed (e.g. 32 bits)	Variable (1–15 bytes on x86)

Instruction count	Small (dozens)	Large (hundreds)
Cycles per instruction	Typically 1 (simple)	Variable, often many
Addressing modes	Few (load/store only)	Many, including memory operands
Register file	Large (16–32+)	Smaller (e.g. 8 on original x86)
Control unit	Hardwired	Microprogrammed
Pipelining	Easy — uniform instructions	Harder — decode complexity
Compiler burden	Heavy — must schedule & fuse	Lighter — CPU does more
Code density	Lower (more instructions)	Higher (complex ops in one op)
Power efficiency	High — mobile/embedded	Lower historically
Typical use	Phones, tablets, Macs, servers, IoT	Desktop/laptop PCs, x86 servers

In practice the line has blurred: modern x86 CPUs decode CISC instructions into RISC-like *micro-ops* that a RISC-style back-end pipelines and executes out-of-order. ARM has gained complex SIMD and crypto instructions. The cultural and power-efficiency differences still matter, though: ARM chips dominate phones and increasingly laptops (Apple Silicon) because their simpler decoders draw less power.

4.7.12 GPUs

GPU (Graphics Processing Unit) — a processor architected for massively data-parallel workloads. Where a CPU has a few (4–64) complex cores with deep pipelines and huge caches, a GPU has thousands of simpler cores grouped into *streaming multiprocessors* (*SMs*), each executing the same instruction on many data items at once (SIMD / SIMT).

Backed by a wide, high-bandwidth memory bus (e.g. 384–1024 bits, hundreds of GB/s–TB/s).

Why it matters

- **Graphics** — rendering an HD frame applies the same shader to millions of pixels. Perfectly parallel; millions of GPU threads nail it.
- **Machine learning** — deep learning is dominated by dense matrix multiplications ($C = A \times B$). Each output element is an independent dot product — exactly the kind of embarrassingly-parallel load a GPU was born for. Modern GPUs add *tensor cores* that do 4×4 matrix-multiply-accumulate in a single instruction.
- **Cryptocurrency mining** — hashing operations (SHA-256, Ethash) are independent arithmetic across trillions of inputs — again, parallel.
- **Scientific computing** — fluid dynamics, molecular dynamics, weather.

GPU vs CPU

	CPU	GPU
Cores	4–64, complex, out-of-order	1,000s, simple, in-order, SIMD
Optimised for	Low-latency serial code, branchy logic	High-throughput data-parallel code
Memory bandwidth	~50–100 GB/s	500 GB/s – 3 TB/s
Cache per thread	Large	Tiny — threads hide latency by switching
Flow control	Handles divergent branches cheaply	Divergent branches serialise warps

When your problem has lots of independent data and little branching, a GPU can be 10–100× faster than a CPU on the same silicon budget. When it has complex control flow (e.g. a compiler), a

CPU still wins.

4.7.13 Input-Output Devices

Input devices turn human-world signals into bits; output devices do the opposite. The OS mediates every transfer through a *device driver* (see 4.6 operating-system notes).

Input devices

- **Keyboard** — a matrix of switches; each key press closes a row/column contact that the keyboard controller scans and reports as a scancode.
- **Mouse** — optical or laser sensor tracks surface texture at 1,000+ dpi, reporting dx/dy deltas and button events.
- **Scanner** — digitises paper into a bitmap, often followed by software recognition.
 - **OCR (optical character recognition)** — turns the image of text into editable characters. Used for book digitisation, receipts, number-plate recognition.
 - **OMR (optical mark recognition)** — detects filled-in bubbles on a pre-printed form. Used for multiple-choice exam papers and ballots.
 - **Barcode scanner** — laser detects the pattern of thick/thin dark bars encoding a product ID. EAN-13 on groceries, QR codes on phones.
- **Microphone, webcam, touch screen, games controller** — other common input transducers.

Output devices

- **Monitor** — LCD or OLED panel with millions of addressable pixels. Image delivered by the GPU over HDMI/DisplayPort at 60–240 frames per second.
- **Printer:**
 - **Inkjet** — sprays microscopic ink droplets; cheap, photo-quality, slow per page.
 - **Laser** — laser charges a drum to attract toner; fast, sharp, ideal for office volume.
 - **3D printer** — builds an object layer by layer by extruding plastic or fusing powder.
- **Speakers, actuators, haptic motors** — convert electrical signals to physical output.

4.7.14 Secondary Storage Devices

Covered in detail under 4.7.9. In exam answers, remember the three families (magnetic, solid-state, optical), at least one physical mechanism per family, and the trade-offs of speed, capacity, cost and durability.

Assembly Language Programming

The AQA A-Level assembly language examined on paper 2 is a simplified ARM-like dialect. You will not be asked to invent a compiler, but you will be asked to read and write short programs using the following instructions.

Instruction reference

Mnemonic	Meaning	Example	Effect
LDR Rd, addr	Load register from memory	LDR R1, 100	$R1 \leftarrow \text{memory}[100]$
STR Rs, addr	Store register to memory	STR R1, 100	$\text{memory}[100] \leftarrow R1$
ADD Rd, Rs, Op2	Add	ADD R1, R2, #3	$R1 \leftarrow R2 + 3$
SUB Rd, Rs, Op2	Subtract	SUB R1, R2, R3	$R1 \leftarrow R2 - R3$
MOV Rd, Op2	Copy value into register	MOV R1, #0	$R1 \leftarrow 0$
CMP Rn, Op2	Compare (set flags from $Rn - Op2$)	CMP R1, #10	update SR flags
B label	Unconditional branch	B loop	$PC \leftarrow \text{loop}$
BEQ label	Branch if equal ($Z=1$)	BEQ done	conditional jump
BNE label	Branch if not equal ($Z=0$)	BNE loop	conditional jump

BGT label	Branch if greater than (signed)	BGT big	conditional jump
BLT label	Branch if less than (signed)	BLT small	conditional jump
AND Rd, Rs, Op2	Bitwise AND	AND R1, R2, #1	isolate LSB
ORR Rd, Rs, Op2	Bitwise OR	ORR R1, R2, #4	set bit 2
EOR Rd, Rs, Op2	Bitwise XOR	EOR R1, R1, R1	clear R1
MVN Rd, Op2	Move NOT (bitwise complement)	MVN R1, R2	$R1 \leftarrow \sim R2$
LSL Rd, Rs, #n	Logical shift left by n	LSL R1, R2, #3	$R1 \leftarrow R2 \times 8$
LSR Rd, Rs, #n	Logical shift right by n	LSR R1, R2, #1	$R1 \leftarrow R2 \div 2$ (unsigned)
HALT	Stop execution	HALT	end of program

Assembly language — a low-level programming language whose mnemonics map one-to-one (approximately) onto a specific processor's machine-code instructions. An *assembler* translates it to machine code. Used when the programmer needs fine-grained control over registers, timing, or memory layout — device drivers, bootloaders, performance-critical inner loops.

Worked programme — sum the integers 1 to 10

We accumulate a running total in R1, use R2 as the loop counter, and compare against 10 in R3. At the end we store the total to memory address 100.

```

MOV    R1, #0           ; total = 0      (accumulator)
MOV    R2, #1           ; counter = 1
MOV    R3, #10          ; upper bound

loop:  ADD    R1, R1, R2   ; total = total + counter
      ADD    R2, R2, #1   ; counter = counter + 1
      CMP    R2, R3       ; compare counter with 10
      BLE    loop         ; if counter <= 10, repeat
                          ; (on an AQA paper, use CMP + BGT done; B loop)
      STR    R1, 100      ; memory[100] = total  (= 55)
      HALT

```

Trace of the accumulator R1 across loop iterations:

```

Iter 1:  R1 = 0  + 1  = 1
Iter 2:  R1 = 1  + 2  = 3
Iter 3:  R1 = 3  + 3  = 6
Iter 4:  R1 = 6  + 4  = 10
Iter 5:  R1 = 10 + 5  = 15
Iter 6:  R1 = 15 + 6  = 21
Iter 7:  R1 = 21 + 7  = 28
Iter 8:  R1 = 28 + 8  = 36
Iter 9:  R1 = 36 + 9  = 45
Iter 10: R1 = 45 + 10 = 55  ← final total

```

If your exam dialect lacks `BLE`, rewrite the tail as:

```

      CMP    R2, R3
      BGT    done        ; exit when counter > 10
      B      loop
done:  STR    R1, 100
      HALT

```

Worked programme — conditional absolute value

```

LDR    R1, 200          ; load x
CMP    R1, #0
BGT    positive         ; if x > 0, skip negation
BEQ    positive         ; if x = 0, also skip
MVN    R1, R1           ; R1 = NOT x
ADD    R1, R1, #1       ; R1 = -x (two's complement)

```

```
positive: STR R1, 201      ; store |x|  
                HALT
```

Common Misconceptions

- **"The PC points to the current instruction."** No — the PC points to the *next* instruction. During execute it has already been incremented.
- **"Cache is a kind of RAM slower than main memory."** The opposite — cache is faster, smaller, more expensive SRAM sitting between the CPU and DRAM.
- **"Pipelining makes a single instruction faster."** It does not; end-to-end latency of any one instruction is unchanged. It improves *throughput* by overlapping multiple instructions.
- **"Virtual memory means the computer has more RAM."** Virtual memory is an *abstraction* that uses secondary storage as overflow; performance drops by orders of magnitude when pages actually live on disk.
- **"RISC is always faster than CISC."** Not inherently; RISC simplifies the *hardware*, shifting complexity to the compiler. For a given silicon budget, RISC tends to be more power-efficient, which matters in phones but less in desktops.
- **"GPUs are just faster CPUs."** They are not. GPUs sacrifice branch handling and cache-per-thread for massive parallelism. On sequential, branchy code they can be *slower* than a CPU.
- **"Machine code and assembly are the same."** Assembly uses mnemonics; machine code is the binary after assembly. The CPU executes machine code only.

Exam-Ready Summary

- A computer is CPU + main memory (RAM/ROM) + I/O + secondary storage, linked by address, data and control buses.
- The stored-program concept (von Neumann): programs and data share memory, both represented in binary, enabling flexible, general-purpose computing.
- CPU internals: ALU, CU, register set, clock, internal buses. Pipelining (3- or 5-stage) overlaps instructions for near one-per-cycle throughput, limited by structural, data (RAW/WAR/WAW) and control hazards; branch prediction mitigates control hazards.

- Key registers: PC (next instruction address), CIR (current instruction), MAR (address bus drive), MBR/MDR (data bus drive), SR (flags Z/N/C/V), ACC, GPRs.
- Instructions = opcode + operand. Addressing modes: immediate, direct, indirect, indexed.
- FDE cycle in register-transfer notation: $MAR \leftarrow [PC]$; $PC \leftarrow [PC] + 1$; $MBR \leftarrow [memory[MAR]]$; $CIR \leftarrow [MBR]$; decode; execute.
- Performance factors: clock speed (power/heat limited), number of cores (Amdahl's law), cache hierarchy (L1/L2/L3 sizes and latencies), word length, address bus width (2^n locations).
- Secondary storage needed for non-volatile persistence and capacity; HDDs (platters/heads/tracks/sectors), SSDs (NAND flash, wear-levelling, TRIM), optical (pits/lands, laser).
- Virtual storage extends RAM onto disk via paging; thrashing is the pathological failure case.
- RISC: simple fixed-length instructions, large register file, load/store, deep pipelines, mobile/embedded (ARM). CISC: complex variable-length instructions, memory operands, microcoded, desktop x86.
- GPU: thousands of SIMD cores, wide memory bus, ideal for graphics, ML matrix multiplication, crypto mining. Wrong tool for branchy serial code.
- I/O devices: keyboard, mouse, scanner (OCR/OMR/barcode) in; monitor, printer out.
- Assembly mnemonics: LDR, STR, ADD, SUB, MOV, CMP, B/BEQ/BNE/BGT/BLT, AND, ORR, EOR, MVN, LSL, LSR, HALT. Know how to sum 1..10 in a loop.